

DE CENT JEUX À LA FOIS

Bruegel peint plus de deux cents enfants, chacun absorbé dans son propre jeu selon ses propres règles, sans chef d'orchestre : un système multi-agent modélise exactement cette idée, une multitude d'entités autonomes dont les interactions locales font émerger, sans plan centralisé, l'ordre collectif de toute la scène.



FIG. XVIII. PIETER BRUEGEL L'ANCIEN, « JEUX D'ENFANTS », 1560

Modélisation à base d'agents (ABM), automates cellulaires, essaims et algorithmes collectifs, plateformes NetLogo/Mesa/GAMA, validation et enjeux d'un modèle multi-agent en géographie, en trois parcours de niveau.

SOMMAIRE

1. ANATOMIE FORMELLE D'UN MODÈLE MULTI-AGENT

2. AUTOMATES CELLULAIRES APPROFONDIS

3. LE MODÈLE DE SCHELLING FORMALISÉ

4. NETLOGO : PRISE EN MAIN DE LA PLATEFORME

5. MESA (PYTHON) : CONSTRUIRE UN MODÈLE AGENT-BASED EN CODE

6. INTELLIGENCE EN ESSAIM (SWARM INTELLIGENCE)

7. LES BOIDS DE REYNOLDS FORMALISÉS

8. APPLICATION GÉOGRAPHIQUE COMPLÈTE : CROISSANCE URBAINE PAR AGENTS

Dans « Jeux d'enfants » (1560), Pieter Brueghel l'Ancien peint plus de deux cents enfants sur une même place, chacun absorbé dans son propre jeu, sa propre règle, sans meneur ni chef d'orchestre — et pourtant l'ensemble forme une scène cohérente, une place de village qui « fonctionne ». C'est exactement le principe d'un système multi-agent : de nombreuses entités autonomes, chacune ne suivant que ses propres règles locales, sans vue d'ensemble ni coordination centrale, dont les interactions produisent malgré tout un

comportement collectif organisé — parfois même un comportement qu'aucun agent, pris isolément, n'a jamais eu l'intention de produire. Ce module explore cette idée depuis son intuition la plus simple (un automate cellulaire, un banc de poissons) jusqu'à sa mise en œuvre rigoureuse en géographie : simulation de croissance urbaine, de trafic, d'épidémie, de foule. Trois pistes complètes ci-dessous (choisis la tienne dans le filtre « Afficher ») : chacune se lit seule, du début à la fin.

SUPÉRIEUR

1. ANATOMIE FORMELLE D'UN MODÈLE MULTI-AGENT

Un modèle multi-agent (agent-based model, ABM) se décompose toujours en trois éléments formels, quelle que soit la plateforme utilisée pour l'implémenter.

ÉLÉMENT	CE QU'IL DÉFINIT
Agents	Des entités autonomes, chacune avec un état interne (position, ressources, statut...) et des règles de comportement qui transforment cet état à chaque pas de temps, en fonction de ce que l'agent perçoit
Environnement	L'espace dans lequel les agents évoluent et interagissent : une grille (comme un automate cellulaire), un réseau (graphe de nœuds et d'arêtes, ex. un réseau routier), ou un espace continu (coordonnées réelles, comme les boids)
Interactions	Les règles qui régissent comment les agents s'influencent entre eux ou avec l'environnement : compétition pour une ressource, transmission d'une information, évitement d'une collision

TYPE D'ENVIRONNEMENT	CE QU'IL PERMET DE REPRÉSENTER
Grille (automate cellulaire)	Un espace discrétisé en cellules régulières — le plus proche d'un raster SIG, adapté à une occupation du sol ou une propagation
Réseau (graphe)	Des nœuds et des arêtes — le plus proche d'un réseau routier ou hydrographique vectoriel, adapté à un déplacement contraint
Espace continu	Des coordonnées réelles, sans découpage en cellules — adapté à un mouvement libre (boids, foule) où la position exacte compte plus que la cellule occupée

REMARQUE

L'ORDONNANCEMENT (SCHEDULER) : UN CHOIX QUI CHANGE LE RÉSULTAT

À chaque pas de temps, dans quel ordre les agents agissent-ils ? Ce choix, souvent négligé, n'est pas neutre. Un ordonnancement séquentiel (chaque agent agit l'un après l'autre, dans un ordre fixe) peut avantager systématiquement les premiers agents de la liste, qui voient l'état déjà modifié par personne alors que les derniers voient un environnement déjà transformé par tous les autres. Un ordonnancement aléatoire (l'ordre est retiré au hasard à chaque pas de temps) élimine ce biais systématique. Un ordonnancement simultané (tous les agents perçoivent le même état gelé, décident, puis toutes les actions s'appliquent ensemble) évite tout effet d'ordre, au prix d'un calcul plus coûteux et de règles de résolution de conflit à prévoir (que se passe-t-il si deux agents veulent la même case libre au même instant ?). Changer uniquement le type d'ordonnancement, toutes règles égales par ailleurs, peut changer significativement le résultat d'une simulation — un piège classique de reproductibilité.

SUPÉRIEUR

2. AUTOMATES CELLULAIRES APPROFONDIS

Le Jeu de la Vie (piste Lycée) est un cas particulier d'une famille plus large : tout automate cellulaire définit une grille, un voisinage, et une règle de transition qui recalcule l'état de chaque cellule à partir de l'état de son voisinage. Deux définitions de voisinage dominent la pratique.

VOISINAGE DE MOORE (8 CELLULES)

- Les 8 cellules qui entourent une cellule, y compris les 4 diagonales
- Utilisé par le Jeu de la Vie classique
- Capture aussi les contacts « en coin », pertinent pour une propagation qui peut sauter en diagonale (feu, épidémie)

VOISINAGE DE VON NEUMANN (4 CELLULES)

- Seulement les 4 cellules directement adjacentes (haut, bas, gauche, droite)

- Plus restrictif : ignore les diagonales
- Souvent plus réaliste pour une propagation strictement le long d'un réseau ou d'une grille orthogonale (ex. un système d'irrigation en damier)

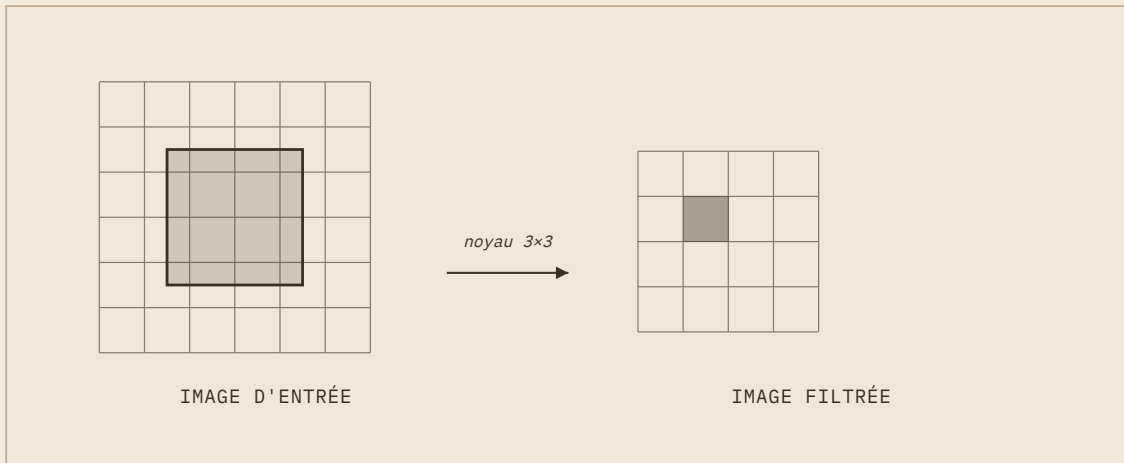


PLANCHE VIII. PAR ANALOGIE : UN NOYAU DE CONVOLUTION GLISSÉ PIXEL PAR PIXEL ILLUSTRE LA MÊME LOGIQUE QU'UNE RÈGLE DE TRANSITION D'AUTOMATE CELLULAIRE, RECALCULÉE À PARTIR DU SEUL VOISINAGE IMMÉDIAT DE CHAQUE CELLULE.

- Automate de feu de forêt : chaque cellule est un arbre sain, en feu, ou brûlé ; une cellule saine prend feu avec une certaine probabilité si au moins une cellule voisine (Moore) est déjà en feu, puis devient brûlée au pas de temps suivant — un automate cellulaire probabiliste, pas strictement déterministe comme le Jeu de la Vie
- Propagation épidémique sur grille : chaque cellule (ou chaque agent occupant une cellule) est saine, infectée ou immunisée (modèle SIR spatialisé) ; une cellule saine s'infecte avec une probabilité qui croît avec le nombre de voisines infectées, illustrant directement comment une épidémie se propage en tache d'huile dans l'espace plutôt qu'instantanément partout

RÈGLE PROBABILISTE DE L'AUTOMATE DE FEU DE FORÊT

$$P(\text{sain} \rightarrow \text{en feu}) = 1 - (1 - p)^n$$

p = probabilité qu'une seule cellule voisine en feu transmette l'incendie à la cellule saine considérée, n = nombre de cellules voisines déjà en feu (voisinage de Moore, donc n entre 0 et 8). Chaque voisine en feu tente indépendamment d'enflammer la cellule ; la probabilité qu'aucune n'y parvienne est $(1 - p)^n$, d'où la probabilité complémentaire ci-dessus. Une cellule entourée de trois foyers voisins ($n = 3$) a donc une probabilité de prendre feu bien supérieure à une cellule qui n'en touche qu'un seul — un effet de masse directement comparable, dans son principe, au seuil de voisins infectés d'une propagation épidémique sur grille.

REMARQUE

L'AUTOMATE CELLULAIRE AU SERVICE D'UNE VRAIE CARTE : CA_MARKOV

Ce que ce module présente ici reste le mécanisme général d'un automate cellulaire, indépendant de toute application. Le module TerrSet applique précisément ce mécanisme à une vraie carte d'occupation du sol dans CA_MARKOV (piste Master/Recherche) : chaque pixel d'une image satellite change de classe selon son état actuel, une carte d'aptitude au changement, et l'état de ses pixels voisins — exactement la même logique de voisinage et de règle de transition, appliquée cette fois à un vrai problème de simulation d'occupation du sol plutôt qu'à une grille abstraite.

Le trafic routier lui-même se prête à un automate cellulaire à une dimension : Nagel et Schreckenberg (1992) modélisent une file de véhicules sur une route à voie unique comme une ligne de cellules, chaque cellule pouvant contenir au plus un véhicule caractérisé par une vitesse entière.

- Accélération : si la vitesse du véhicule est inférieure à la vitesse maximale et à la distance libre devant lui, elle augmente de 1
- Freinage : si la distance libre devant le véhicule est inférieure à sa vitesse actuelle, sa vitesse est ramenée à cette distance (pour ne pas percuter le véhicule suivant)
- Freinage aléatoire : avec une petite probabilité p , la vitesse diminue encore de 1, même sans obstacle — cette seule règle stochastique suffit à faire apparaître des embouteillages qui se propagent vers l'arrière sans aucune cause extérieure (accident, feu rouge)
- Avancée : chaque véhicule avance enfin d'autant de cellules que sa vitesse finale

EXEMPLE

LE BOUCHON FANTÔME, UN COMPORTEMENT ÉMERGENT DOCUMENTÉ

Le résultat le plus cité du modèle de Nagel-Schreckenberg est de reproduire numériquement le « bouchon fantôme » (phantom traffic jam) : un ralentissement qui se forme et se propage vers l'arrière du flux sans qu'aucun accident, péage ou obstacle ne l'explique — la seule règle de freinage aléatoire, appliquée à de nombreux véhicules, suffit à le faire émerger, exactement le même principe d'émergence que le Jeu de la Vie ou Schelling, appliqué cette fois à un flux de trafic à une dimension.

ATTENTION

LE PARADOXE DE BRAESS : UN AUTRE RÉSULTAT CONTRE-INTUITIF DU TRAFIC SIMULÉ

Un second résultat célèbre de la modélisation du trafic, documenté par Dietrich Braess (1968) et depuis reproduit dans de nombreuses simulations multi-agents de réseau routier, montre qu'ajouter une route supplémentaire à un réseau peut, dans certaines configurations, augmenter le temps de trajet moyen de tous les usagers plutôt que le réduire : chaque conducteur, en choisissant égoïstement le trajet le plus rapide pour lui à cet instant (un comportement d'agent parfaitement rationnel et local), peut collectivement aboutir à un équilibre plus lent pour tout le monde que si cette route n'avait jamais existé. Un exemple de plus, appliqué au trafic, du thème central de ce module : un comportement individuellement rationnel n'aboutit pas nécessairement à un optimum collectif.

REMARQUE

LA CLASSIFICATION DE WOLFRAM : QUATRE GRANDES FAMILLES DE COMPORTEMENT

Stephen Wolfram (« A New Kind of Science », 2002) a proposé, à partir de l'étude systématique des automates cellulaires les plus simples possibles (une seule ligne de cellules, deux états, voisinage à 3 cellules — les « automates cellulaires élémentaires »), une classification en quatre classes de comportement qui se retrouve aussi, en pratique, sur des automates bidimensionnels plus riches comme le Jeu de la Vie : Classe I, tout converge rapidement vers un état uniforme et figé (extinction) ; Classe II, le système se stabilise en motifs périodiques simples, stables ou oscillants ; Classe III, le comportement reste chaotique et imprévisible en permanence, sans structure durable ; Classe IV, la plus riche et la plus rare, produit des structures complexes qui persistent, interagissent et se déplacent sans jamais se figer ni sombrer dans le chaos total — c'est précisément dans cette classe IV que se trouvent le Jeu de la Vie et ses planeurs. Cette classification aide à situer d'emblée un nouvel automate cellulaire géographique (feu de forêt, épidémie) : un automate qui s'éteint toujours (classe I) ou qui reste toujours chaotique (classe III) est en général moins informatif pour une simulation qu'un automate en classe IV, capable de motifs spatiaux durables et interprétables.

3. LE MODÈLE DE SCHELLING FORMALISÉ

Formellement, chaque agent i occupe une case d'une grille et appartient à l'un de deux groupes (ou davantage). Il calcule à chaque pas de temps la proportion de ses voisins immédiats (voisinage de Moore, typiquement) qui appartiennent au même groupe que lui, et compare cette proportion à un seuil de tolérance t fixé pour toute la simulation.

RÈGLE DE SATISFACTION DE SCHELLING

$\text{satisfait}(i) = [\text{proportion de voisins du même groupe que } i] \geq t$

Si $\text{satisfait}(i)$ est faux, l'agent i déménage vers une case libre choisie au hasard sur la grille (ou vers la case libre la plus proche, selon la variante). La simulation s'arrête quand plus aucun agent n'est insatisfait, ou après un nombre de pas de temps fixé.

EXEMPLE

SENSIBILITÉ AU PARAMÈTRE DE TOLÉRANCE

Faire varier uniquement t change radicalement le résultat final. Avec $t = 0$ (aucune exigence), la grille reste proche de sa configuration initiale aléatoire, sans ségrégation notable. Avec t autour de 0,3 à 0,4 (une préférence très modérée, loin d'être une hostilité), la grille converge presque toujours vers de larges zones homogènes bien séparées — la ségrégation collective mesurée (par exemple la proportion moyenne de voisins du même groupe, une fois la simulation stabilisée) dépasse largement le seuil de tolérance individuel qui l'a pourtant produite. C'est l'expérience de calibration la plus instructive du modèle : rejouer la simulation pour plusieurs valeurs de t et comparer la ségrégation résultante.

SEUIL DE TOLÉRANCE T	RÉSULTAT TYPIQUE OBSERVÉ APRÈS STABILISATION
0,0 – 0,1	Grille proche du désordre initial, mélange quasi aléatoire des deux groupes
0,3 – 0,4	Ségrégation collective forte : larges zones homogènes, malgré une exigence individuelle modérée
0,6 et au-delà	Ségrégation quasi totale, souvent atteinte en très peu de déménagements

SUPÉRIEUR

4. NETLOGO : PRISE EN MAIN DE LA PLATEFORME

NetLogo (Wilensky, Northwestern University, depuis 1999) est l'environnement de modélisation multi-agent le plus utilisé en enseignement et en recherche exploratoire, pour sa prise en main rapide. Le monde y est une grille de « patches » (cellules fixes de l'environnement, comme les cases d'un automate cellulaire) sur laquelle se déplacent des « turtles » (agents mobiles, malgré leur nom, qui peuvent représenter n'importe quoi — un piéton, un véhicule, un ménage).

- Interface : une vue du monde en grille, des curseurs (sliders) pour ajuster les paramètres du modèle sans toucher au code, des boutons pour lancer/mettre en pause la simulation
- Langage NetLogo : un langage de haut niveau, proche de l'anglais courant, conçu pour qu'un utilisateur non-programmeur puisse lire et modifier un modèle existant
- Bibliothèque de modèles intégrée (Models Library) : des dizaines de modèles ABM classiques prêts à l'emploi, dont une implémentation du modèle de Schelling et du Jeu de la Vie

PSEUDO-NETLOGO : UNE RÈGLE DE PROPAGATION SIMPLIFIÉE

```
to go ask turtles [ if any? neighbors with [infected?] [ set
infected? true ] ] tick end
```

`ask turtles [...]` applique le bloc de code à chaque agent tour à tour (ordonnancement séquentiel par défaut de NetLogo, modifiable) ; `neighbors` renvoie les patches ou turtles voisins (Moore par défaut) ; `tick` avance l'horloge de simulation d'un pas. Cette syntaxe très lisible — proche du pseudo-code — est la raison principale du succès pédagogique de NetLogo.

PSEUDO-NETLOGO : LE MODÈLE DE SCHELLING

```
to go ask turtles with [not satisfied?] [ move-to one-of
patches with [not any? turtles-here] ] ask turtles [ let
voisins (turtles-on neighbors) let semblables count voisins
with [groupe = [groupe] of myself] set satisfied? (semblables
/ (count voisins + 1)) >= seuil-tolerance ] tick end
```

`seuil-tolerance` est exposé comme un curseur (slider) réglable directement dans l'interface, sans toucher au code : c'est ce qui permet, dans NetLogo, de rejouer immédiatement l'expérience de sensibilité au paramètre t décrite en section 3, sans reprogrammer quoi que ce soit.

SUPÉRIEUR

5. MESA (PYTHON) : CONSTRUIRE UN MODÈLE AGENT-BASED EN CODE

Mesa est un framework Python de modélisation multi-agent, plus verbeux que NetLogo mais entièrement intégré à l'écosystème Python (NumPy, pandas, GeoPandas) déjà rencontré dans ce site — un choix naturel dès qu'un modèle doit s'articuler avec de vraies données géographiques. Un modèle Mesa s'organise systématiquement autour de deux classes.

- Classe Agent : définit l'état d'un agent (attributs) et sa méthode `step()`, exécutée à chaque pas de temps — l'équivalent direct de la règle de comportement d'un agent
- Classe Model : instancie la grille d'environnement (MultiGrid si plusieurs agents peuvent partager une case, SingleGrid si une case n'accueille qu'un agent à la fois, comme dans Schelling), crée la population d'agents, et définit sa propre méthode `step()` qui fait avancer tous les agents d'un pas de temps selon l'ordonnancement choisi
- DataCollector : objet qui enregistre, à chaque pas de temps, une ou plusieurs statistiques (nombre d'agents satisfaits, proportion infectée...) pour analyse et visualisation après la simulation, sans avoir à instrumenter soi-même chaque boucle

SQUELETTE MINIMAL D'UN AGENT MESA

```
class ResidentAgent(Agent):
    def step(self):
        voisins = self.model.grid.get_neighbors(self.pos, moore=True)
        meme_groupe = sum(1 for v in voisins if v.groupe == self.groupe)
        if len(voisins) and meme_groupe / len(voisins) < self.seuil:
            self.model.grid.move_to_empty(self)
```

Directement l'équivalent Python de la règle de satisfaction de Schelling formalisée en section 3 : `get_neighbors(moore=True)` applique un voisinage de Moore, `move_to_empty` déplace l'agent vers une case libre s'il est insatisfait.

LA CLASSE MODEL QUI L'ENTOURE

```
class SchellingModel(Model):
    def __init__(self, largeur, hauteur, seuil):
        self.grid = SingleGrid(largeur, hauteur, torus=False)
        self.schedule = RandomActivation(self)
        self.datacollector = DataCollector(
            model_reporters={"pct_satisfaits": lambda m: part_satisfaits(m)}
        )
        # création et placement aléatoire des agents ResidentAgent...
    def step(self):
        self.datacollector.collect(self)
        self.schedule.step()
```

`RandomActivation` applique un ordonnancement aléatoire (section 1) plutôt que séquentiel, pour éviter tout biais systématique lié à l'ordre de création des agents. `datacollector.collect(self)` est appelé à chaque pas de temps, avant que les agents n'agissent, pour enregistrer l'état du système et pouvoir tracer, après coup, l'évolution de la proportion d'agents satisfaits — exactement la statistique dont dépend l'expérience de sensibilité au seuil t de la section 3.

6. INTELLIGENCE EN ESSAIM (SWARM INTELLIGENCE)

Au-delà de l'imitation d'un comportement biologique (boids), certains algorithmes d'essaim exploitent directement le comportement émergent d'agents simples pour résoudre un problème d'optimisation — trouver un chemin, un minimum, une meilleure configuration — sans qu'aucun agent n'ait de vision globale du problème.

L'optimisation par colonies de fourmis (Ant Colony Optimization, ACO, Dorigo, 1992) s'inspire directement du comportement réel des fourmis : chaque fourmi explore le graphe (un réseau de chemins possibles) en déposant une trace de phéromone sur le chemin qu'elle emprunte, cette phéromone s'évaporant progressivement avec le temps.

MISE À JOUR DE PHÉROMONE (ACO)

$$\tau_{ij}(t+1) = (1 - \rho) \cdot \tau_{ij}(t) + \sum_k \Delta\tau_{ij}^k$$

τ_{ij} = quantité de phéromone sur l'arête entre les nœuds i et j , ρ = taux d'évaporation (entre 0 et 1), $\Delta\tau_{ij}^k$ = phéromone déposée par la fourmi k sur cette arête, généralement inversement proportionnelle à la longueur totale du chemin qu'elle a suivi (un chemin court reçoit plus de phéromone qu'un chemin long). Au fil des itérations, les chemins courts accumulent plus de phéromone (empruntés plus souvent, donc renforcés plus souvent) tandis que l'évaporation efface progressivement les chemins longs délaissés : le chemin le plus court émerge sans qu'aucune fourmi n'ait jamais eu connaissance du graphe entier.

L'optimisation par essaims particulaires (Particle Swarm Optimization, PSO, Kennedy & Eberhart, 1995) s'inspire du vol groupé (proche des boids) plutôt que des fourmis : chaque particule explore un espace de solutions possibles avec une position et une vitesse, ajustées à chaque itération en fonction de la meilleure position qu'elle a personnellement trouvée et de la meilleure position trouvée par l'essaim entier.

MISE À JOUR DE VITESSE (PSO)

$$v_i(t+1) = w \cdot v_i(t) + c_1 r_1 \cdot (p_i - x_i(t)) + c_2 r_2 \cdot (g - x_i(t))$$

v_i = vitesse de la particule i , x_i = sa position actuelle, p_i = la meilleure position qu'elle a personnellement rencontrée, g = la meilleure position rencontrée par l'essaim entier, w = coefficient d'inertie (garde une part du mouvement précédent), c_1 et c_2 = poids respectifs de l'attraction vers le meilleur personnel et le meilleur collectif, r_1 et r_2 = nombres aléatoires entre 0 et 1 (introduisent une exploration non déterministe). La position se met ensuite à jour simplement par $x_i(t+1) = x_i(t) + v_i(t+1)$.

REMARQUE

UN POINT COMMUN : JAMAIS DE CARTE GLOBALE

Ni une fourmi ni une particule ne connaît la structure entière du problème qu'elle contribue à résoudre. ACO n'a de la carte que ce que chaque fourmi individuelle en a parcouru ; PSO n'a du paysage de solutions que ce que chaque particule a personnellement exploré, complété par le seul résumé (le meilleur point trouvé) partagé avec l'essaim. C'est la même logique d'émergence que le reste de ce module, appliquée à un problème d'optimisation plutôt qu'à une simulation.

ALGORITHME	TYPE DE PROBLÈME	APPLICATION GÉOGRAPHIQUE TYPIQUE
ACO	Un chemin optimal sur un graphe (problème combinatoire, ex. le voyageur de commerce)	Optimiser une tournée de collecte de déchets ou une tournée de livraison sur un réseau routier réel
PSO	Trouver le minimum ou le maximum d'une fonction continue (problème d'optimisation continue)	Calibrer automatiquement les paramètres libres d'un modèle spatial (par exemple les poids d'une AHP, module Le Compas) en minimisant l'écart à un motif observé

SUPÉRIEUR

7. LES BOIDS DE REYNOLDS FORMALISÉS

Les trois règles de Reynolds (piste Lycée) se formalisent chacune comme un vecteur de correction de vitesse, calculé à partir des seuls voisins situés dans un rayon de perception r autour de l'agent. La vitesse résultante à chaque pas de temps est une somme pondérée des trois vecteurs.

MISE À JOUR DE VITESSE D'UN BOID

$$v(t+1) = v(t) + w_s \cdot \text{Séparation} + w_a \cdot \text{Alignement} + w_c \cdot \text{Cohésion}$$

Séparation = vecteur qui éloigne l'agent de ses voisins trop proches (souvent pondéré inversement à la distance, pour repousser plus fortement un voisin très proche) ; Alignement = différence entre la vitesse moyenne des voisins et la vitesse actuelle de l'agent ; Cohésion = vecteur dirigé vers le centre de masse (le barycentre des positions) des voisins. w_s , w_a , w_c sont des poids réglables : augmenter w_s produit un vol plus dispersé, augmenter w_c produit un vol plus compact — la même vitesse résultante est ensuite bornée par une vitesse maximale, sans quoi le vecteur de cohésion peut faire diverger la simulation si deux groupes de boids sont très éloignés l'un de l'autre.

SUPÉRIEUR

8. APPLICATION GÉOGRAPHIQUE COMPLÈTE : CROISSANCE URBAINE PAR AGENTS

Un modèle multi-agent de croissance urbaine typique combine directement plusieurs notions déjà vues dans ce site : chaque agent est un ménage ou un promoteur qui choisit une parcelle à urbaniser selon une carte d'aptitude (distance aux routes, pente, occupation du sol actuelle — les mêmes couches qu'une évaluation multicritère, module TerrSet), tandis que l'environnement lui-même est une grille raster réelle chargée depuis un SIG plutôt qu'une grille abstraite.

- L'occupation du sol initiale (module Le Compas, statistiques de zone) fournit l'état de départ de la grille
- Le réseau routier (module TerrSet, section MCE) fournit un critère d'accessibilité qui pondère l'attractivité de chaque cellule pour un nouvel agent
- Chaque agent (ménage) applique une règle de choix locale, comparable à la règle de satisfaction de Schelling : occuper la cellule libre la plus attractive dans son rayon de recherche, plutôt que la cellule globalement optimale sur toute la carte
- Répété sur de nombreux pas de temps, ce processus fait émerger un patron spatial de croissance urbaine (extension continue le long des axes routiers, ou au contraire mitage dispersé) sans qu'aucun agent n'ait planifié la forme finale de la ville

REMARQUE

VARIANTE : LA DIFFUSION SPATIALE D'UNE INNOVATION

Le même schéma agent + environnement SIG s'applique à la diffusion d'une innovation dans l'espace (une nouvelle pratique agricole, l'équipement en panneaux solaires) plutôt qu'à la croissance urbaine. Le modèle de diffusion de Bass (1969), à l'origine non spatial, distingue deux mécanismes d'adoption : une adoption par innovation propre (un agent adopte spontanément, indépendamment de ses voisins) et une adoption par imitation (un agent adopte d'autant plus vite qu'une grande proportion de ses voisins a déjà adopté). Rendre ce modèle spatialement explicite revient à remplacer la population globale du modèle de Bass par des agents individuels situés sur une vraie carte, où le terme d'imitation ne dépend plus de la proportion adoptée dans la population entière mais seulement du voisinage local de chaque agent — un patron de diffusion en tache d'huile émerge alors depuis les premiers foyers d'adoption, comparable dans sa forme à la propagation épidémique sur grille de la section 2.

DEVOIR À RENDRE : MODÈLE SUR PAPIER OU TABLEUR

FAIRE TOURNER UN SCHELLING SIMPLIFIÉ À LA MAIN

Sur une grille 6×6, place au hasard 12 agents du groupe A, 12 agents du groupe B, et laisse 12 cases vides. Fixe un seuil de tolérance $t = 0,4$. Applique la règle de satisfaction à chaque agent (voisinage de Moore, bords traités comme absents plutôt que cycliques) et fais migrer chaque agent insatisfait vers une case vide choisie au hasard, pas de temps par pas de temps, pendant 3 pas de temps. Mesure la proportion moyenne de voisins du même groupe au début et à la fin.

- Le voisinage de Moore est correctement identifié pour les cellules de bord (moins de 8 voisins réels)
- La règle de satisfaction est appliquée à tous les agents insatisfaits avant qu'aucun ne déménage (simultané), pas les uns après les autres avec l'état déjà modifié
- La ségrégation mesurée en fin de simulation est bien supérieure au seuil de tolérance individuel de 0,4, illustrant l'écart entre préférence individuelle et résultat collectif

- Bilan — à retenir : un ABM formel = agents (état + règles) + environnement (grille/réseau/continu) + interactions + ordonnancement (séquentiel/aléatoire/simultané, jamais neutre) ; voisinage de Moore (8) contre von Neumann (4) ; Schelling se formalise par un seuil de tolérance comparé à la proportion de voisins semblables ; NetLogo (patches/turtles, langage haut niveau) contre

Mesa (classes Agent/Model, DataCollector, Python) ; ACO exploite dépôt/évaporation de phéromone, PSO combine meilleure position individuelle et collective ; les boids se formalisent en trois vecteurs de correction pondérés ; un modèle de croissance urbaine articule ces règles sur de vraies couches SIG.

VOIR AUSSI : VOIR L'APPLICATION GÉOSPATIALE : CA_MARKOV

Le module TerrSet applique le mécanisme général d'automate cellulaire présenté ici à une vraie simulation d'occupation du sol, couplée à une chaîne de Markov.

VOIR AUSSI : REVOIR LES DONNÉES MOBILISÉES : OCCUPATION DU SOL ET RÉSEAU ROUTIER

Le module Le Compas détaille les statistiques de zone et les opérations spatiales qui alimentent la carte d'aptitude d'un modèle de croissance urbaine.